Міністерство освіти і науки України Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського” Факультет інформатики та обчислювальної техніки Кафедра інформаційних систем та технологій

Лабораторна робота №2

із дисципліни «**Технології розроблення програмного забезпечення**»

Тема **«Діаграма варіантів використання. Сценарії варіантів використання. Діаграма UML. Діаграми класів. Концептуальна модель системи»**

Виконав:

студент групи ІА–24

Яблонський Д.Б.

Перевірив:

Мягкий М.Ю.

Київ 2024

Мета: Вивчення основних принципів побудови та проектування програмних систем з використанням схем прецедентів, діаграм класів та структури бази даних.

Теоретичні відомості

Прецеденти (Use Case Diagram)

Діаграма прецедентів використовується для візуалізації функціональних вимог до системи. Вона показує, як користувачі (актор або актори) взаємодіють із системою через певні сценарії використання (прецеденти). Основними елементами діаграми є актори, прецеденти, а також зв'язки між ними. Прецеденти допомагають виявити основні функції системи та забезпечують їх розуміння на високому рівні.

Діаграма класів (Class Diagram)

Діаграма класів використовується для моделювання статичної структури системи. Вона показує класи, атрибути класів, методи (операції), а також зв'язки між класами, такі як асоціації, агрегації, композиції та успадкування. Класи представляють основні компоненти системи, їхні характеристики (атрибути) та поведінку (методи). Зв'язки показують, як ці класи взаємодіють між собою.

База даних та її структура

База даних — це організований набір даних, які зберігаються в структурованому вигляді, зазвичай у вигляді таблиць. Таблиці в базі даних складаються з рядків (записів) і стовпців (полів), що містять атрибути даних. Структура бази даних визначає, як дані взаємопов'язані між собою. Основними елементами є таблиці, ключі (первинні та зовнішні) і зв'язки між таблицями (один-до-одного, один-до-багатьох, багато-до-багатьох).

Шаблон Репозиторію (Repository Pattern)

Шаблон Репозиторію використовується для абстрагування доступу до даних. Він дозволяє взаємодіяти з базою даних через клас-репозиторій, що інкапсулює всі операції збереження, отримання, оновлення та видалення даних. Це забезпечує слабку залежність між бізнес-логікою та логікою доступу до даних, роблячи систему більш гнучкою для змін або модернізацій.

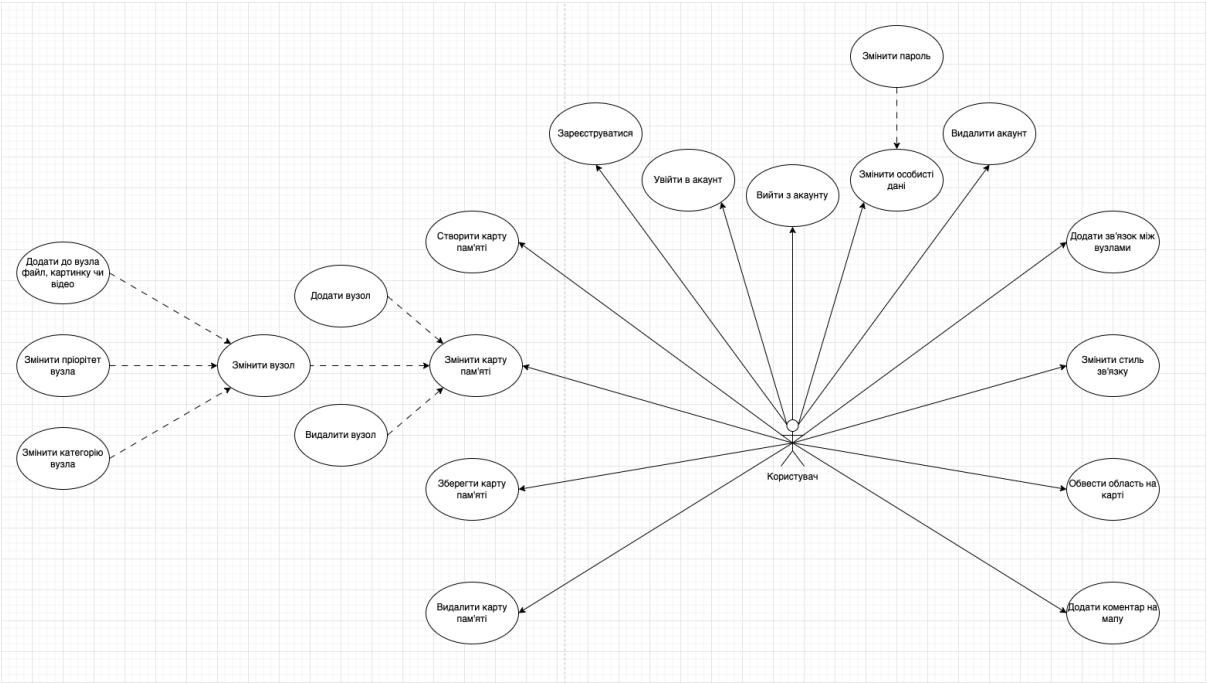
Вибір прецедентів та аналіз

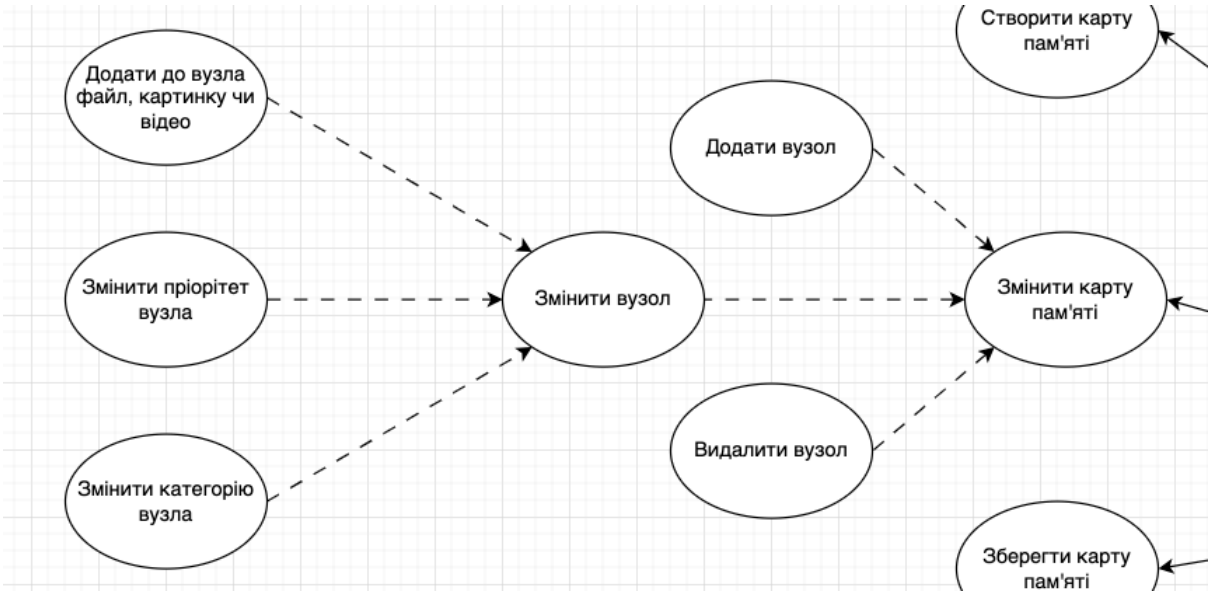
Прецеденти дозволяють ідентифікувати сценарії використання, що найбільш підходять для вашої системи. Важливо вибрати і проаналізувати кілька подібних систем, щоб побачити, як їх функціональність відповідає вимогам вашого проекту. Це допомагає розробити найбільш ефективні та зручні рішення для користувачів.

Хід роботи

1. Ознайомитися з короткими теоретичними відомостями.
2. Проаналізуйте тему та намалюйте схему прецеденту, що відповідає обраній темі лабораторії.
3. Намалюйте діаграму класів для реалізованої частини системи.
4. Виберіть 3 прецеденти і напишіть на їх основі прецеденти.
5. Розробити основні класи і структуру системи баз даних.
6. Класи даних повинні реалізувати шаблон Репозиторію для взаємодії з базою даних.

Діаграма прецедентів





Реєстрація та керування обліковим записом

- Зареєструватися: Користувач може створити новий обліковий запис.
- Увійти в акаунт: Користувач може увійти в систему, використовуючи облікові дані.

- Вийти з акаунту: Користувач може вийти зі свого облікового запису.
- Змінити пароль: Користувач може змінити свій поточний пароль.
- Змінити особисті дані: Користувач може змінювати особисту інформацію (наприклад, ім'я, email).
- Видалити акаунт: Користувач може видалити свій обліковий запис.

Управління картами пам'яті

- Створити карту пам'яті: Користувач може створити нову карту для організації ідей.
- Змінити карту пам'яті: Користувач може редагувати існуючу карту пам'яті.
- Зберегти карту пам'яті: Користувач може зберегти поточну карту на диск або в базу даних.
- Видалити карту пам'яті: Користувач може видалити карту пам'яті.

Управління вузлами на карті пам'яті

- Додати вузол: Користувач може додати новий вузол на карту пам'яті.
- Змінити вузол: Користувач може редагувати зміст вузла.
- Видалити вузол: Користувач може видалити вузол з карти пам'яті.
- Додати до вузла файл, картинку чи відео: Користувач може прикріпити медіафайли до вузла.
- Змінити пріоритет вузла: Користувач може змінювати пріоритетність вузла (наприклад, відмічати важливі вузли).
- Змінити категорію вузла: Користувач може змінювати категорію для кращої організації вузлів.

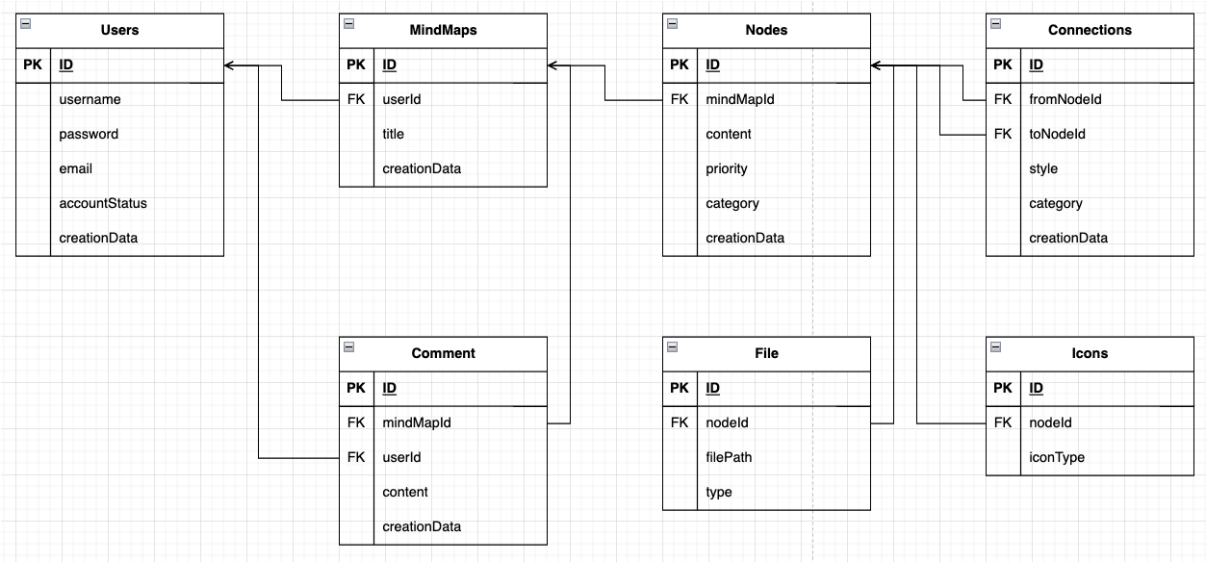
Робота зі зв'язками між вузлами

- Додати зв'язок між вузлами: Користувач може створити зв'язок між двома вузлами.
- Змінити стиль зв'язку: Користувач може змінювати візуальний стиль з'єднань між вузлами (наприклад, суцільні лінії, пунктирні).

Додаткові можливості

- Обвести область на карті: Користувач може виділити певну область на карті пам'яті для організації або групування вузлів.
- Додати коментар на карту: Користувач може залишати коментарі на карті пам'яті для додаткових пояснень або зауважень.

Діаграма класів



Users (Користувачі)

- ID: Унікальний ідентифікатор користувача (первинний ключ).
- username: Логін користувача.
- password: Пароль користувача.
- email: Електронна пошта.
- accountStatus: Статус облікового запису (наприклад, "active" або "deleted").
- creationData: Дата створення облікового запису.

MindMaps (Карти пам'яті)

- ID: Унікальний ідентифікатор карти пам'яті (первинний ключ).
- userId: Ідентифікатор користувача (зовнішній ключ до Users).
- title: Назва карти пам'яті.
- creationData: Дата створення карти пам'яті.

Nodes (Вузли)

- ID: Унікальний ідентифікатор вузла (первинний ключ).
- mindMapId: Ідентифікатор карти пам'яті, до якої належить вузол (зовнішній ключ до MindMaps).
- content: Вміст вузла (текст або дані).
- priority: Пріоритет вузла.
- category: Категорія вузла.
- creationData: Дата створення вузла.

Connections (Зв'язки)

- ID: Унікальний ідентифікатор зв'язку (первинний ключ).
- fromNodeId: Ідентифікатор початкового вузла (зовнішній ключ до Nodes).
- toNodeId: Ідентифікатор кінцевого вузла (зовнішній ключ до Nodes).
- style: Стиль зв'язку (тип лінії, пунктир тощо).
- category: Категорія зв'язку.
- creationData: Дата створення зв'язку.

File (Файли)

- ID: Унікальний ідентифікатор файлу (первинний ключ).
- nodeId: Ідентифікатор вузла, до якого прикріплено файл (зовнішній ключ до Nodes).
- filePath: Шлях до файлу.
- type: Тип файлу (зображення, відео).

Icons (Іконки)

- ID: Унікальний ідентифікатор іконки (первинний ключ).
- nodeId: Ідентифікатор вузла, до якого прикріплено іконку (зовнішній ключ до Nodes).
- iconType: Тип іконки (категорія або терміновість).

Comment (Коментарі)

- ID: Унікальний ідентифікатор коментаря (первинний ключ).
- mindMapId: Ідентифікатор карти пам'яті (зовнішній ключ до MindMaps).
- userId: Ідентифікатор користувача, який залишив коментар (зовнішній ключ до Users).
- content: Текст коментаря.
- creationData: Дата створення коментаря.

Зв'язки між сутностями:

- Users має зв'язок один до багатьох з MindMaps (один користувач може створити кілька карт пам'яті).
- MindMaps має зв'язок один до багатьох з Nodes (одна карта пам'яті

може мати кілька вузлів).
- MindMaps має зв'язок один до багатьох з Comment (одна карта може мати кілька коментарів).
- Nodes має зв'язок один до багатьох з File (один вузол може мати кілька прикріплених файлів).
- Nodes має зв'язок один до багатьох з Icons (один вузол може мати кілька іконок).
- Nodes має зв'язок один до багатьох з Connections (один вузол може мати кілька з'єднань з іншими вузлами).

Висновок: У ході виконання даної лабораторної роботи було опрацьовано основи моделювання програмних систем з використанням діаграм прецедентів та діаграм класів. Ми розробили структуру бази даних для системи карт пам'яті, яка включає сутності користувачів, карт пам'яті, вузлів, зв'язків між ними, файлів, іконок та коментарів.

В цій роботі ми створили діаграми прецедентів, які описують взаємодію користувача із системою та побудували діаграму класів, що відображають зв'язки між сутностями бази даних.

Отримані результати дозволяють краще зрозуміти процес моделювання програмного забезпечення і забезпечити ефективну взаємодію між різними компонентами системи. Такий підхід допомагає створювати гнучкі, масштабовані та підтримувані рішення для управління даними.